

01 – Arrays Functions JS

1. Métodos de acceso y búsqueda

- **f at()**

Recibe un valor numérico entero y devuelve el elemento en esa posición, permitiendo valores positivos y negativos. Los valores negativos contarán desde el último elemento del array.

```
let letras = ['a', 'b', 'c', 'd'];

console.log(letras.at(1)); // 'b'
console.log(letras.at(-1)); // 'd' (índice negativo cuenta desde el final)
console.log(letras); // ['a', 'b', 'c', 'd'] (no cambia)
```

- **f concat()**

Se usa para unir dos o más arrays. Este método no cambia los arrays existentes, sino que devuelve un nuevo array.

```
let num1 = [1, 2];
let num2 = [3, 4];
let combinado = num1.concat(num2, [5, 6], 7);

console.log(combinado); // [1, 2, 3, 4, 5, 6, 7]
console.log(num1); // [1, 2] (no cambia)
```

- **f copyWithin()**

Transfiere una copia plana de una sección a otra dentro del mismo array (o contexto similar), sin modificar su propiedad length y lo devuelve.

```
let arr = [1, 2, 3, 4, 5];
arr.copyWithin(0, 3);
console.log(arr); // [4, 5, 3, 4, 5]
```

- **f entries()**

El método entries() retorna un nuevo objeto Array Iterator que contiene los pares clave/valor para cada índice de la matriz.

```
let frutas = ['manzana', 'banana', 'cereza'];
for (let [i, fruta] of frutas.entries()) {
  console.log(`Índice ${i} → ${fruta}`);
}
// Índice 0 → manzana
```

```
// Índice 1 → banana
// Índice 2 → cereza
```

- **f every()**

El método every() de las instancias de Array prueba si todos los elementos del arreglo pasan la prueba implementada por la función proporcionada. Devuelve un valor booleano.

```
let edades = [18, 20, 25, 30];
console.log(edades.every(edad => edad >= 18)); // true
console.log(edades.every(edad => edad >= 21)); // false
```

- **f fill()**

El método fill() cambia todos los elementos en un arreglo por un valor estático, desde el índice start (por defecto 0) hasta el índice end (por defecto array.length). Devuelve el arreglo modificado.

```
let arr = [1, 2, 3];
arr.fill(0, 1, 3);
console.log(arr); // [1, 0, 0]
```

- **f filter()**

Crea un nuevo array con todos los elementos que cumplan la condición implementada por la función dada.

```
let numeros = [1, 2, 3, 4, 5, 6];
let pares = numeros.filter(n => n % 2 === 0);
console.log(pares); // [2, 4, 6]
console.log(numeros); // [1, 2, 3, 4, 5, 6] (original intacto)
```

- **f find()**

Devuelve el valor del primer elemento del array que cumple la función de prueba proporcionada.

```
let personas = [{nombre: 'Ana', edad: 17}, {nombre: 'Luis', edad: 21}];
let adulto = personas.find(p => p.edad >= 18);
console.log(adulto); // {nombre: 'Luis', edad: 21}
```

- **f findIndex()**

Devuelve el índice del primer elemento de un array que cumpla con la función de prueba proporcionada. En caso contrario devuelve -1.

```
let valores = [3, 5, 8, 12];
console.log(valores.findIndex(v => v > 10)); // 3
```

- **f findLast()**

Devuelve el último elemento que cumple la condición.

```
let lista2 = [1, 4, 7, 10, 7];  
console.log(lista2.findLastIndex(n => n === 7)); // 4
```

- **f findLastIndex()**

Devuelve el índice del último elemento que cumple la condición.

```
let arr = [5, 12, 8, 14];  
console.log(arr.findLastIndex(n => n > 10)); // 3
```

2. Transformación y manipulación

- **f flat()**

Crea una nueva matriz con todos los elementos de sub-array concatenados recursivamente hasta la profundidad especificada.

```
let anidado = [1, [2, [3, [4]]]];  
console.log(anidado.flat(2)); // [1, 2, 3, 4]
```

- **f flatMap()**

Mapea y aplana un nivel en una sola operación.

```
let nums = [1, 2, 3];  
console.log(nums.flatMap(n => [n, n * 2])); // [1, 2, 2, 4, 3, 6]
```

- **f forEach()**

Ejecuta la función indicada una vez por cada elemento del array.

```
[1, 2, 3].forEach(n => console.log(n * 2));  
// 2  
// 4  
// 6
```

- **f includes()**

Determina si una matriz incluye un determinado elemento, devuelve true o false según corresponda.

```
const array1 = [1, 2, 3];  
  
console.log(array1.includes(2));  
// Expected output: true  
  
const pets = ["cat", "dog", "bat"];  
  
console.log(pets.includes("cat"));
```

```
// Expected output: true

console.log(pets.includes("at"));
// Expected output: false
```

- **f indexOf()**

Retorna el primer índice en el que se puede encontrar un elemento dado en el array, o retorna -1 si el elemento no está presente.

```
var array = [2, 9, 9];
array.indexOf(2); // 0
array.indexOf(7); // -1
```

- **f join()**

Convierte el arreglo en cadena.

```
const elements = ["Fire", "Air", "Water"];

console.log(elements.join());
// Expected output: "Fire,Air,Water"

console.log(elements.join(""));
// Expected output: "FireAirWater"

console.log(elements.join("-"));
// Expected output: "Fire-Air-Water"
```

- **f keys()**

Devuelve un nuevo objeto Array Iterator que contiene las claves para cada índice en el arreglo.

```
let arr = ["a", "b"];
for (let key of arr.keys()) {
  console.log(key);
}
// 0
// 1
```

- **f lastIndexOf()**

Devuelve el último índice en el que un cierto elemento puede encontrarse en el arreglo, ó -1 si el elemento no se encontrara.

```
let arr = [1, 2, 1, 2];
console.log(arr.lastIndexOf(2)); // 3
```

3. Orden y modificación directa

- **f map()**

Crea un nuevo arreglo aplicando una función.

```
let cuadrados = [1, 2, 3].map(n => n ** 2);  
console.log(cuadrados); // [1, 4, 9]
```

- **f pop()**

Elimina el último elemento de un array y lo devuelve. Este método cambia la longitud del array.

```
let arr = [1, 2];  
console.log(arr.pop()); // 2  
console.log(arr); // [1]
```

- **f push()**

Añade uno o más elementos al final de un array y devuelve la nueva longitud del array.

```
let cola = [1, 2];  
cola.push(3, 4);  
console.log(cola); // [1, 2, 3, 4]
```

- **f reduce()**

Ejecuta una función reductora sobre cada elemento de un array, devolviendo como resultado un único valor.

```
let suma = [1, 2, 3, 4].reduce((acc, val) => acc + val, 0);  
console.log(suma); // 10
```

- **f reduceRight()**

Igual que reduce pero de derecha a izquierda.

```
let arr = [[0, 1], [2, 3]];  
console.log(arr.reduceRight((a, b) => a.concat(b))); // [2, 3, 0, 1]
```

- **f reverse()**

Invierte el orden de los elementos de un array in place. El primer elemento pasa a ser el último y el último pasa a ser el primero.

```
let orden = [1, 2, 3];  
orden.reverse();  
console.log(orden); // [3, 2, 1]
```

- **f shift()**

Elimina el primer elemento del array y lo retorna. Este método modifica la longitud del array.

```
let fila = ['A', 'B', 'C'];
let primero = fila.shift();
console.log(primero); // 'A'
console.log(fila);    // ['B', 'C']
```

- **f slice()**

Devuelve una copia de una parte del array dentro de un nuevo array empezando por inicio hasta fin (fin no incluido). El array original no se modificará.

```
let colores = ['rojo', 'verde', 'azul'];
colores.splice(1, 1, 'amarillo', 'morado');
console.log(colores); // ['rojo', 'amarillo', 'morado', 'azul']
```

- **f some()**

Comprueba si al menos un elemento del array cumple con la condición implementada por la función proporcionada.

```
const array = [1, 2, 3, 4, 5];

// Checks whether an element is even
const even = (element) => element % 2 === 0;

console.log(array.some(even));
// Expected output: true
```

- **f sort()**

Ordena los elementos de un arreglo (array) localmente y devuelve el arreglo ordenado.

```
const frutas = ["guindas", "manzanas", "bananas"];
frutas.sort(); // ['bananas', 'guindas', 'manzanas']

const puntos = [1, 10, 2, 21];
puntos.sort(); // [1, 10, 2, 21]
// Tenga en cuenta que 10 viene antes que 2
// porque '10' viene antes que '2' según la posición del valor Unicode.

const cosas = ["word", "Word", "1 Word", "2 Words"];
cosas.sort(); // ['1 Word', '2 Words', 'Word', 'word']
```

- **f splice()**

Cambia el contenido de un array eliminando elementos existentes y/o agregando nuevos elementos.

```
const months = ["Jan", "March", "April", "June"];
months.splice(1, 0, "Feb");
// Inserts at index 1
console.log(months);
// Expected output: Array ["Jan", "Feb", "March", "April", "June"]

months.splice(4, 1, "May");
// Replaces 1 element at index 4
console.log(months);
// Expected output: Array ["Jan", "Feb", "March", "April", "May"]
```

4. Conversión y utilidades

- **f toLocaleString()**

Devuelve una cadena de texto representando los elementos del array. Los elementos son convertidos a texto usando su método toLocaleString y dichos Strings son separados por un caracter específico para la localidad (como una coma para la separación de decimales ",").

```
let precios = [1234.56, 7890.12];
console.log(precios.toLocaleString('es-ES', { style: 'currency', currency: 'EUR' }));
// "1.234,56 €", "7.890,12 €"
```

- **f toString()**

Devuelve una cadena de caracteres representando el array especificado y sus elementos.

```
const array1 = [1, 2, "a", "1a"];
console.log(array1.toString());
// Expected output: "1,2,a,1a"
```

- **f unshift()**

Agrega uno o más elementos al inicio del array, y devuelve la nueva longitud del array.

```
const array1 = [1, 2, 3];

console.log(array1.unshift(4, 5));
// Expected output: 5

console.log(array1);
// Expected output: Array [4, 5, 1, 2, 3]
```

- **f values()**

Devuelve un nuevo objeto Array Iterator que contiene los valores para cada índice del array.

```
let letras3 = ['x', 'y', 'z'];
for (let valor of letras3.values()) {
  console.log(valor);
}
// x
// y
// z
```